

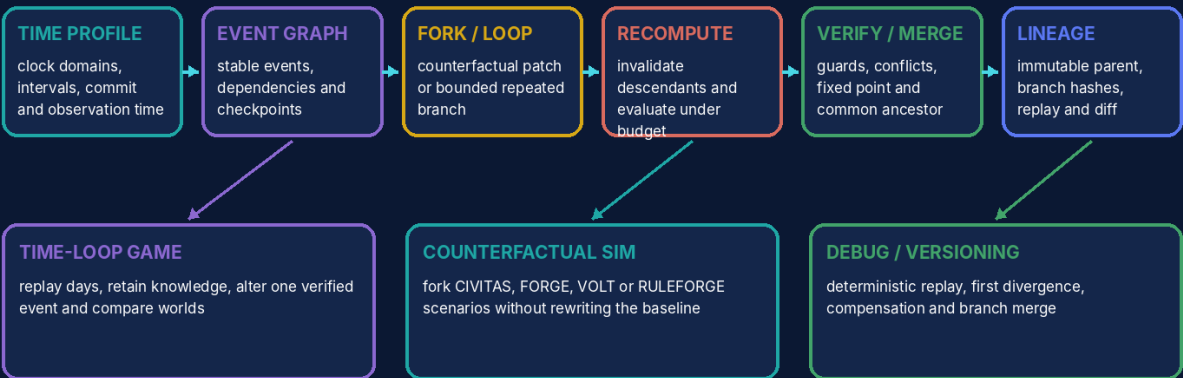
UNIVERSAL GEOMETRIC TOPOLOGICAL
OPEN MODULAR SUBSTRATE

UGTOMS CHRONOFORGE 1.0

Branching Timelines, Time Loops, Counterfactual Worlds and
Replay Cartridge

UGTOMS CHRONOFORGE 1.0 Architecture

Past edits fork replayable branches; loops, merges and counterfactuals are explicit event-graph operations.



Time travel stays fiction; branch identity, event order, uncertainty and irreversible novelty stay mathematically honest.

Pure domain evaluation may propose evidence; only the retained authority chain may commit state.

ugtoms.domain.chronoforge@1.0.0

Component	CHRONOFORGE
Canonical identity	ugtoms.domain.chronoforge@1.0.0
Component type	Temporal domain cartridge
Version / date	1.0.0 / 30 August 2026
Document type	Open modular cartridge specification and implementation-ready profile
Status	Implementation-ready contract with deterministic branch fixtures; no physical time-travel or causal-truth claim

ATTRIBUTION AND EVIDENCE BOUNDARY

Prepared for Tom Klootwijk. Requester attribution is recorded as supplied and is not independently verified. This is a technical design artifact, not legal proof, scientific validation, regulatory approval or production safety certification.

Contents and Reading Rule

Section	Purpose
1	Executive decision and temporal fantasy
2	Source basis and evidence discipline
3	Open modular cartridge ABI
4	Timeline profiles and formal temporal state
5	Clock domains, intervals and event identity
6	Partial order, dependencies and future-cone queries
7	Counterfactual branch fork and descendant invalidation
8	Reversibility, compensation and irreversible novelty
9	Loops, fixed points, progress and finite budgets
10	Branch merge, paradox classes and same-time conflicts
11	Verified timeline patch and immutable lineage
12	CIVITAS city-signal counterfactual demonstration
13	Queries, packing and external adapters
14	Reference validation, boundaries and kill criteria
15	Complete 48-mechanism catalog
16	Claims ledger
17	High-impact profiles and final definition
READING RULE	
The UGTOMS source line supplies the reusable authority, typing, order, evidence and packing discipline. The specialist domain model in this cartridge is an engineering-derived extension. Where the source corpus is silent, this report says so instead of presenting general domain knowledge as source-derived.	

Executive Decision

DECISION - GO

Replace the previously planned generic CAUSAL cartridge with CHRONOFORGE. It keeps the useful dependency discipline but turns it into something actually exciting: immutable timelines, counterfactual forks, time-loop games, reversible and irreversible events, paradox reason codes, bounded recomputation and branch merges.

CHRONOFORGE is the bridge between deterministic replay and full time-manipulation mechanics. It can power a Groundhog-Day game, a branching cinematic story, a counterfactual city simulator or a debugger that literally asks: which earlier event made this world different?

Temporal superpower	CHRONOFORGE contract
Edit the past without lying	A verified past edit creates a new branch from a checkpoint; the parent history remains immutable.

Temporal superpower	CHRONOFORGE contract
Recompute the future	Affected descendants are invalidated and deterministically re-evaluated under explicit models and budgets.
Build honest time loops	Loop entry, state projection, fixed point, progress measure, exit and iteration limit are profile data.
Separate inverse from compensation	Some changes can be reversed exactly; others require a new compensating event or remain irreversible.
Merge worlds carefully	Three-way merge uses a common ancestor, dependency context, conflict policy and a valid non-merge result.
Replay everything	Branch hashes, checkpoints, event order, external novelty and first divergence remain inspectable.

Source Basis and Evidence Discipline

The cartridge inherits a common UGTOMS rule: domain knowledge may be broad, but every authoritative change stays typed, ordered, evidence-bounded and replayable.

- Definitions, operators and profiles live inside the substrate with stable IDs, explicit dependencies and content hashes [S1-S3].
- Pure evaluation and model output remain upstream of support, compatibility, guards, verified proposals, deterministic commit and lineage [S1, S2, S4-S7].
- Coordinates, glyphs, pictures, meshes, simulations, scans and packed keys are not identity or authority by themselves [S1, S4-S10].
- Exactness, uncertainty, capability, budgets and error margins are explicit; UNKNOWN and REJECTED are valid outcomes [S1, S2, S5, S8-S10].
- Component-scoped identities keep this cartridge from masquerading as the newest universal version of every UGTOMS track [S11].
- Existing specialist cartridges remain peers and adapters; this report does not silently absorb their schemas or claims [S13-S17].

ENGINEERING-DERIVED DOMAIN CONTENT

The timeline, counterfactual, loop, reversal, merge and paradox records are new temporal-domain engineering. The corpus contributes time/phase/winding, explicit order, dependencies, deterministic replay, checkpoints, conflict resolution, bounded grammar and certificates. This report does not claim physical time travel or a universal theory of causation.

ID	Registered source	Use in this cartridge	Pages
S1	UGTS_FOUNDATION_UNICODE_v5.pdf	Foundation 5.0 OperatorCells: literal, glyph, semantic, field and converse faces remain co-addressed but typed separately.	pp. 1-21
S2	UGTS_KC_4_2_General_Operator_Order_Addendum(1).pdf	Typed OperatorSpec, dependency/phase/event/replay order, exactness, capability, budgets and adapter fallback policy.	pp. 2-32
S3	UGTS_KC_3_6_Tom_Klootwijk.pdf	Definitions as content-addressed substrate data, resolvable references, dependency closure and explicit operation order.	pp. 1-15
S4	Unified_Geometric_Topological_Substrate_GPU_Native_Addendum.pdf	Canonical support → compatibility → guard → verified event → transition → lineage authority and packed-state boundaries.	pp. 2-15
S5	UGTS_KC_2_0_Tom_Klootwijk_Expanded_Substrate.pdf	Patterns, fields, topology, SE(2)/SE(3), bounded dynamics, hybrid events and numerical/error contracts.	pp. 3-15
S6	UGTS_KC_Two_Hands_3_0_Interactive_Graphics_Runtime.pdf	Stable scene identity, proposals, deterministic commit, checkpoints, rollback, replay and divergence detection.	pp. 3-19
S7	UGTS_KC_3_9_KC_Elizabeth_Vector_Game_Runtime.pdf	Versioned projects, deterministic fixed-step runtime, snapshots/hashes and self-contained browser delivery.	pp. 2-16
S8	UGTS_KC_3_6_2_SCLP_Tom_Klootwijk.pdf	Time/phase/winding, finite topology maps, interval guards, finite grammar and exact packed-key boundaries.	pp. 1-15
S9	UGTS_KC_4_0_Spatial_Evidence_Ledger(1).pdf	Spatial observation versus authority, stable nodes/edges, uncertainty, route topology, repeat-scan change and replay.	pp. 3-15
S10	UGTS_KC_4_1_1_ATLAS_SPATIAL_SEED_SLAM_POCO_X7_PRO_MERGE_REPORT(1).pdf	Camera/IMU proposal path, bounded SLAM front end, stable IDs, ordered verifier and KSEED seed-plus-evidence boundary.	pp. 2-14
S11	UGTS_Versioning_Charter_Phase_1_Foundation_Chronology(1).pdf	Component-scoped identities and explicit release graphs instead of one misleading global ladder.	pp. 1-6
S12	UGTS_KC_Definition_Breaker_DB_0_1_Concept(1).pdf	Rules as physical objects, safe preview, hinges, winding, replay, seeded grammar and reason-coded commit.	pp. 2-24
S13	04_UGTOMS_RULEFORGE_1.0.pdf	Reusable rule-world host: typed actions, visibility, chance, conflict resolution, terminality and deterministic replay.	pp. 2-18
S14	06_UGTOMS_PROOFWEAVE_1.0.pdf	Proof graphs, small kernel, scoped certificates, trust boundaries and proposal-only search.	pp. 2-18

ID	Registered source	Use in this cartridge	Pages
S15	03_UGTOMS_FORGE_1_0.pdf	Parts, interfaces, datum frames, assembly order, inspection and infrastructure/fabrication boundaries.	pp. 2-16
S16	05_UGTOMS_VOLT_1_0.pdf	Typed networks, ports, capacities, events, faults, model adapters and physical-evidence separation.	pp. 2-17
S17	02_UGTOMS_SONORA_1_0.pdf	Executable notation, exact rational timing, cyclic structures, voices and downstream audiovisual projections.	pp. 2-15

Open Modular Cartridge ABI

The cartridge is a concrete instance of the shared domain object D_CHRONOFORGE:

D = (T, O, R, U, I, Q, E, G, Delta, L, Cert, K, Pi)	
Field	Contract
T	Entity, value and state types
O	Typed operators and transformations
R	Relations, graphs, topology and converse pairs
U	Units, dimensions, coordinate and profile conventions
I	Invariants, conservation laws and constraints
Q	Dependency, precedence, phase and reduction order
E	Observations, evidence, uncertainty and provenance
G	Guards, thresholds and admission predicates
Delta	Proposed and committed state patches
L	Lineage, replay, history and migration
Cert	Exactness, residual, interval, scope and status certificates
K	Cold atlas, hot codebook and packed representation
Pi	Optional visual, audio, physical or device projection

A conforming cartridge contains a manifest, type registry, operator registry, relation atlas, unit/profile system, dependency DAG, guard library, transition definitions, claims ledger, examples and validation fixtures. External mature libraries may sit behind adapters when they are safer or more accurate than a fresh reimplementation.

CHRONOFORGE cartridge package

Package record	Required role
timeline_profile.json	Linear, branching, looped or versioned-scenario semantics; clock and edit policies.
event_graph.json	Stable events, dependency edges, temporal intervals, commit order and external novelty.
branches.json	Parent checkpoint, fork patch, frontier, invalidation set, status and content hash.
loop_profiles.json	Entry/exit, state projection, progress measure, fixed-point test and budgets.
reversibility_registry.json	Exact inverse, compensating action, checkpoint rollback and irreversible classes.
merge_profiles.json	Common-ancestor, priority, compatibility, conflict and explicit non-merge semantics.
timeline_ledger.jsonl	Forks, merges, rollbacks, compensations, accepted/rejected patches and replay.
certificates/	Trace, branch-diff, loop, merge and replay certificates.

General Operator Record and Order

```
OperatorSpec = (id, version, syntax, arity, domains, codomain, units, shape,
               laws, exactness, effects, capabilities, dependencies,
               order_contract, invariants, provenance, content_hash)
```

The record follows the General Operator and Order layer [S2]. A symbol or glyph is not allowed to smuggle in undeclared type, unit, order, inverse or authority semantics.

EXAMPLE RECORD

Operator CHR020 counterfactual_proposal
 domain: BranchState x PastEventPatch x RecomputeProfile → BranchCandidate
 preconditions: checkpoint exists, target event supported, dependencies resolve
 effects: proposal only; parent branch immutable
 result: fork checkpoint, invalidation set, recompute plan, budgets and UNKNOWN status
 authority: branch becomes authoritative only after verified fork commit

Normative phase schedule

Phase	Meaning
1-3	parse, normalize and resolve content-addressed references
4-6	type, shape and unit validation; dependency closure
7-9	construct candidates; apply declared composition and reduction order
10-12	evaluate exact or bounded results; issue capability and residual certificate
13	support, compatibility and guard classification
14	verified proposal and deterministic conflict resolution
15	atomic commit, lineage, replay and optional projection

Timeline Profiles and Formal Temporal State

H = (P, K, E, D, B, R, G, Delta, L, Cert)
 P timeline, clock, uncertainty and edit profiles
 K checkpoints and canonical snapshots
 E stable events with observation, event, arrival and commit time
 D explicit dependency/partial-order graph
 B branches, frontiers, invalidation sets and unknown descendants
 R reversibility, loop and merge registries
 G support, conflict, fixed-point and budget guards
 Delta atomic timeline/branch patches
 L immutable parent links, compensation, replay and diff lineage
 Cert trace, loop, merge, branch-diff and replay certificates

Timeline profile	Semantics	Typical use
LINEAR	One ordered commit history; past state queried by checkpoint/replay.	ordinary simulation, logs, debugging
BRANCHING	Verified past edit forks a child branch; parents remain immutable.	counterfactuals, narrative worlds, planning
LOOPED	A declared state projection and transition repeats under entry/exit/fixed-point/budget rules.	time-loop games, cyclic processes
VERSIONED SCENARIO	Named scenarios share definitions/checkpoints but carry separate proposals and outcomes.	CIVITAS, FORGE, VOLT and RULEFORGE studies
MULTI-CLOCK	Several clock domains map through explicit synchronization/uncertainty relations.	distributed simulations, media timelines, sensors

TEMPORAL IDENTITY

A timestamp is not an event identity. Branch ID, clock domain, event ID, dependency frontier, state hash and lineage remain separate.

Clock Domains, Intervals and Event Identity

Temporal field	Meaning
Observation time	When an observer, sensor, log or player became aware of a fact.
Event time	When the modeled domain transition occurred under its clock/profile.
Arrival time	When a message, effect, projectile, packet or proposal reached its consumer.
Commit time	When the authoritative ledger accepted the patch.
Duration/interval	Closed or profile-bound time range with unit, uncertainty and inclusivity.
Clock domain	Simulation tick, wall clock, musical beat, device clock, game turn or narrative phase.
Synchronization relation	Offset, scale, drift, uncertainty and capability between clock domains.
<pre>EventRecord = (event_id, branch_id, clock_domain, event_time_interval, observation_time_interval, arrival_time_interval, commit_sequence, dependencies, requested_patch, pre_hash, post_hash, lineage)</pre>	
UNCERTAINTY RULE	
When temporal intervals overlap, a before/after claim that depends on a strict order remains UNKNOWN unless another declared relation resolves it. Sorting by a point estimate cannot erase uncertainty.	

Partial Order, Dependencies and Future-Cone Queries

$e_i < e_j$ only when one declared relation establishes it:
 dependency(e_i , e_j)
 commit_sequence(e_i) < commit_sequence(e_j)
 non-overlapping event intervals under one clock map
 explicit domain rule

future_cone(e) = descendants_D(e) constrained by branch frontier and profile

Query	Result
ancestors(event)	Exact dependency closure needed to justify or replay the event.
descendants(event)	Events/states that become stale after an edited ancestor.
future_cone(event, budget)	Bounded affected subgraph, unknown adapters and cutoff status.
first_divergence(branch_A, branch_B)	Common ancestor plus first event/state hash where branches differ.
why_before(A,B)	Dependency, interval, commit order or explicit rule establishing order - or UNKNOWN.
simultaneity_class(events)	Events sharing a resolution class before deterministic conflict policy.

NO DEPENDENCY, NO AUTOMATIC CAUSE

CHRONOFORGE records declared dependency and replay consequences. It does not infer real-world causation merely from earlier time, proximity or correlation.

Counterfactual Branch Fork and Descendant Invalidation

CHRONOFORGE branching timeline graph

Editing an earlier event forks a new branch; the original history remains addressable and replayable.

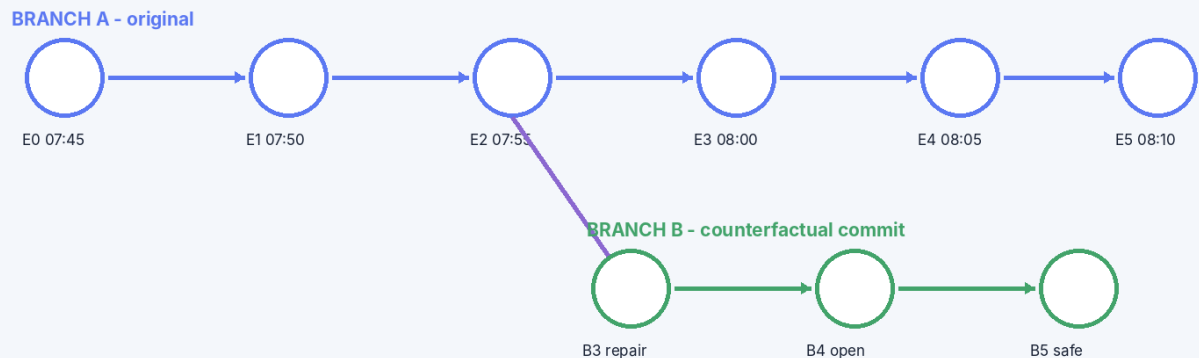


Figure 1. Editing event E2 forks Branch B; original Branch A stays addressable and replayable.

1. Select an exact parent branch and checkpoint before the edited event.
2. Resolve the target event, dependencies, world/domain profile and edit authorization.
3. Create a counterfactual proposal; do not modify the parent branch.
4. Compute the descendant invalidation set from the explicit dependency graph.
5. Apply the proposed past patch in a new branch and assign a new branch identity.
6. Recompute descendants in deterministic order under declared models and budgets.
7. Leave unsupported, ambiguous or budget-exhausted descendants UNKNOWN.
8. Commit the fork, branch frontier, event hashes, invalidations and lineage atomically.

Branch status	Meaning
CANDIDATE	Fork and recompute plan exists but has not passed all guards.
PARTIAL	Some descendants recomputed; others are UNKNOWN, blocked or beyond budget.
CONSISTENT	All required descendants under the selected scope replay and pass invariants.
CONFLICTED	Dependency, same-time, resource, invariant or adapter conflict remains unresolved.
MERGED	A verified merge patch has been committed into a child branch.
RETRACTED	Branch remains in lineage but is no longer an active accepted scenario.

Reversibility, Compensation and Irreversible Novelty

Class	Definition	Example
Exact inverse	A declared inverse returns the same modeled state and required history inside its domain.	Pure coordinate transform or reversible game toggle.
Checkpoint rollback	Discard branch-local descendants and resume from a stored snapshot.	Retry a puzzle loop before external choices are accepted.
Compensating event	A new event offsets an earlier effect while preserving that both occurred.	Refund, repair, heal, reopen or restore service.
Irreversible transition	External novelty, destruction, information disclosure or profile rule forbids exact undo.	A player learned a secret; a real measurement was taken; a file was published.
Unknown reversibility	Adapter or domain profile cannot certify inverse/compensation behavior.	External simulation state without a replay contract.
<pre>reverse(event) -> exact_inverse checkpoint_rollback compensating_event irreversible unknown</pre> A compensation appends history. It never edits the old event out of existence.		
MEMORY MATTERS		
A time-loop game may reset world state while preserving player knowledge as explicit cross-loop lineage. That knowledge is external novelty relative to the reset state and must not be reconstructed from the seed.		

Loops, Fixed Points, Progress and Finite Budgets



Figure 2. A loop profile declares entry, repeated transition, state projection, progress/fixed-point checks and exit.

Loop field	Contract
Entry guard	Exact event/state under which the loop branch begins.
State projection	Which parts must match for loop equivalence; omitted state stays visible as outside the projection.
Transition body	Ordered event graph or rule-world step evaluated each iteration.
Progress measure	Well-founded or bounded metric, or explicit declaration that progress is not guaranteed.
Fixed-point guard	State projection, dependency context and hashes satisfy declared equality/tolerance.
Exit/escape	Guard, choice, failure, budget or external event that terminates the loop.
Budgets	Iterations, events, branches, memory, time and external calls.
LOOP NON-CLAIM	
Returning to the same coordinate, clock reading or image is not enough. A fixed point is profile-bound and must name the compared state, dependencies, uncertainty and history.	

Branch Merge, Paradox Classes and Same-Time Conflicts

Conflict / paradox class	Meaning	Resolution options
Missing cause	A descendant requires an ancestor removed or invalidated by the fork.	recompute, replace dependency, mark unknown, reject branch
Self-invalidating event	An edited event removes the condition that authorized/created itself.	profile-specific fixed point, compensation, conflict, reject
Duplicate authority	Two branches claim incompatible unique object, owner, resource or terminal state.	priority, explicit merge rule, choose branch, non-merge
Unresolved cycle	Dependency loop lacks initialization/convergence semantics.	declare loop profile or reject
Same-time patch conflict	Simultaneous proposals write incompatible values or consume the same resource.	stable priority, commutative batch, compatible merge, conflict event
Temporal uncertainty conflict	Overlapping intervals make required order ambiguous.	seek evidence, widen UNKNOWN, branch alternatives
<pre>merge(base, left, right, profile) -> compare stable IDs and event ancestry -> classify compatible edits, same-effect duplicates and conflicts -> verify dependency closure and invariants -> emit merged child branch OR explicit non-merge certificate</pre>		
NON-MERGE IS VALID		
A merge algorithm is not required to invent a compromise. When rules, identities or irreversible events conflict, a verified non-merge certificate is the honest result.		

Verified Timeline Patch and Immutable Lineage

```

user / model / debugger / game / planner
-> temporal proposal
-> branch + checkpoint + event support
-> dependency, clock, profile and authorization compatibility
-> interval, conflict, reversibility, invariant and budget guards
-> descendant invalidation + recompute trace
-> verified fork / rollback / loop / compensation / merge proposal
-> atomic timeline patch
-> branch content hash + pre/post hashes + immutable parent link
-> replay, diff, visualization or downstream simulation

```

Patch type	Atomic contents
Fork	new branch ID, parent checkpoint, past event patch, invalidation set, frontier and lineage
Rollback	checkpoint selection, discarded branch-local descendants, reason and active-frontier change
Compensation	new event, offset effect, residuals, dependencies and both old/new lineage
Loop update	iteration event, projected state, progress/fixed-point status, budget and exit status
Merge	common ancestor, accepted edits, conflicts, resulting child branch and certificate
Retraction	active-status transition, reason and preserved immutable history
PARENT IMMUTABILITY	
A child branch may supersede an active scenario, but it may not rewrite the bytes or hashes of its parent timeline. Correction is another event, branch or retraction in lineage.	

CIVITAS City-Signal Counterfactual Demonstration

CHRONOFORGE city-signal counterfactual

A CIVITAS signal event is edited in the past; descendants are recomputed in a new branch instead of rewriting history.

TIME	EVENT / STATE	BRANCH	STATUS
07:50	signal inspection omitted	original	accepted event
07:55	gate remains closed	original	derived state
08:00	vehicle queue conflict	original	incident patch
07:48	maintenance proposal	branch B	counterfactual fork
07:50	inspection passes	branch B	verified event
07:55	gate opens	branch B	recomputed descendant
08:00	queue clears	branch B	no incident

Branch A remains immutable; Branch B carries the counterfactual patch, new hashes and new lineage.

Figure 3. A past maintenance event creates Branch B; original Branch A and its incident remain intact.

Operation	Deterministic result
Load Branch A checkpoint at 07:45.	State hash H0 and three scheduled descendant events resolve.
Propose maintenance at 07:48.	Fork support and authorization pass; parent remains unchanged.
Edit 07:50 inspection outcome to PASS.	New event ID and patch committed inside Branch B.
Invalidate descendants.	Gate-closed and queue-conflict descendants marked stale by dependency closure.
Recompute 07:55.	Gate-open event generated under the same signal and schedule profile.
Recompute 08:00.	Queue remains below conflict guard; incident event is absent in Branch B.
Compare branches.	First divergence = 07:50; Branch A retains incident; Branch B records avoided incident.
Replay both.	Each branch independently reproduces its own frontier hash.

DEMO BOUNDARY

This is a counterfactual model. It proves only that two declared deterministic branches differ under the supplied rules. It does not establish what would have happened in reality.

Queries, Packing and External Adapters

Query	Returns
trace(event)	ancestors, dependencies, clock relations, branch, patch, hashes and replay status
future_cone(event, budget)	invalidated descendants, recompute plan, unknown adapters and cutoff
branch_diff(A,B)	common ancestor, first divergence, event/state changes, unknowns and active frontier
loop_status(loop_id)	iteration, projected state, progress, fixed-point/exit and remaining budgets
merge_status(base,L,R)	compatible edits, conflicts, invariant failures and merge/non-merge certificate
why_not_reversible(event)	profile, missing inverse, external novelty, compensation option or unknown status
Packing strategy	Boundary
Checkpoint + event deltas	Exact when transitions and external novelty are fully recorded.
Shared definition/event DAG	Identical immutable nodes may be content-addressed once; branch order and references remain explicit.
Seeded scenario reconstruction	Only declared deterministic models and schedules reconstruct; human choices and real input remain novelty.
Compact branch/frontier codes	Locate branch state but do not replace event graph, uncertainty or lineage.
Pruned history view	May hide old detail for presentation; authoritative archive and hash commitments remain policy-bound.
Adapter	Role
Version control	Map commits/branches/merges/reverts into typed event and lineage profiles.
RULEFORGE / games	Provide turn, state and action events for time-loop, rewind and branching-world mechanics.
CIVITAS / simulation	Fork a city/world checkpoint and compare service, route and incident consequences.
PROOFWEAVE	Verify scoped invariants, fixed-point claims or merge certificates where a formal proof object exists.
Media / animation / SONORA	Synchronize beat, frame, cue and event clocks through explicit clock-domain maps.
Debug/runtime	Replay, checkpoint, rollback, first divergence and deterministic trace; no hidden state repair.

Reference Validation, Boundaries and Kill Criteria

REFERENCE SELF-CHECK RESULT

32/32 deterministic specification self-checks passed. These checks cover catalog uniqueness, required fields, dependency closure, demo arithmetic and declared reason codes. They are package evidence, not independent domain validation.

Checked area	Result
Catalog integrity	48 unique CHR IDs; every mechanism has family, name and core contract.
Parent immutability	Forking from a past event leaves the original branch bytes and hashes unchanged.
Invalidation fixture	Editing the 07:50 event marks only dependency descendants stale before recomputation.
Branch replay	Original and counterfactual branches each reproduce their own frontier hash.
Reversibility fixture	Exact inverse, compensation, checkpoint rollback and irreversible classes remain distinct.
Loop fixture	Fixed-point comparison uses declared state projection and terminates on exit or iteration budget.
Merge fixture	Compatible edits merge; conflicting unique state emits an explicit non-merge certificate.
Temporal uncertainty	Overlapping intervals return UNKNOWN for strict order claims.

Explicit non-claims

- No physical time travel, retrocausality, spacetime manipulation or universal causal theory is claimed.
- No counterfactual simulation proves what would have happened in reality.
- No equal timestamp, coordinate, render or repeated state projection establishes complete timeline identity.
- No exact inverse, branch merge, fixed point, convergence or deterministic reconstruction is assumed without a declared profile.
- No content hash proves causal truth; it proves record integrity only.
- No external version-control, simulation, narrative, media or proof-system compatibility is claimed until an adapter is tested.

Kill criteria

9. Reject or narrow a use when a symbol, codebook slot or packed record cannot resolve to its exact global definition.
10. Reject when a capability or exactness label is stronger than the model, measurement or external source supports.
11. Reject when units, profiles, coordinate systems, operand order or dependency order are implicit.
12. Reject when a proposal bypasses support, compatibility, guards, deterministic commit or lineage.
13. Reject when an external adapter is required for safety or conformance but its version, calibration or provenance is absent.
14. Reject any performance, compression or safety claim without a named workload, target, error/equivalence contract and evidence.

Complete CHRONOFORGE 1.0 Mechanism Catalog

The catalog contains 48 cartridge-local mechanisms. These IDs are component-scoped and do not silently extend a global M-number ladder.

ID	Family	Mechanism	Core contract
CHR001	Profile	Timeline profile	Declares linear, branching, looped or versioned-scenario semantics plus allowed edit, merge and replay operations.
CHR002	Time	Clock domain	Named monotonic, simulation, turn, media, wall or logical clock with unit, origin and synchronization policy.
CHR003	Time	Time coordinate	Typed value in a clock domain; identical numbers in different clocks are not equal events.
CHR004	Time	Duration and interval	Closed/open interval with unit, uncertainty and ordering semantics.
CHR005	Identity	Event identity	Stable event ID, definition, branch, parents, proposal/commit times and lineage.
CHR006	Identity	Branch identity	Stable branch ID and fork point remain part of world identity even when state snapshots match.
CHR007	Identity	Snapshot identity	Canonical state hash plus profile, branch, event frontier and external novelty context.
CHR008	Time	Observation time	When evidence was observed; distinct from when the described event occurred.
CHR009	Time	Event time	Profile-bound time at which an event is modeled to take effect.
CHR010	Time	Arrival and commit time	When a proposal reaches authority and when its patch commits; lateness is explicit.
CHR011	Order	Before/after relation	Strict or non-strict temporal order inside a declared clock/profile.
CHR012	Order	Simultaneity class	Events share one ordered resolution class without assuming physical simultaneity.
CHR013	Dependency	Event dependency edge	Parent event/state condition required for a descendant definition or patch.
CHR014	Dependency	Future-cone query	Finite descendants whose validity may change after an ancestor edit.
CHR015	Dependency	Ancestor closure	Exact dependency closure used to justify or replay one event.
CHR016	Dependency	Descendant invalidation	Marks derived events stale after an edited ancestor until deterministically recomputed.
CHR017	Branch	Branch fork	Creates a new branch from a verified checkpoint and counterfactual patch; parent branch is immutable.
CHR018	Branch	Branch selection	Chooses an active branch for query/presentation without deleting alternatives.
CHR019	Branch	Branch merge	Three-way or policy-bound merge with common ancestor, conflict trace and explicit non-merge result.
CHR020	Branch	Counterfactual proposal	Proposes a past-state change and declares the earliest affected checkpoint and recompute budget.
CHR021	Guard	Dependency support	The edited event lies in the selected branch/time support and its required parent state exists.
CHR022	Reversibility	Reversibility profile	Declares exact inverse, compensating action, checkpoint-only rollback or irreversible status.
CHR023	Reversibility	Inverse patch	Exact state inverse under a declared profile; not inferred from a reversed description.
CHR024	Reversibility	Compensating event	New event that offsets an earlier effect without pretending history was erased.
CHR025	Reversibility	Irreversible marker	Declares external novelty, destructive transition or policy boundary that forbids simple rollback.

ID	Family	Mechanism	Core contract
CHR026	Replay	Checkpoint	Serialized state, branch frontier, hashes and external references at a safe replay boundary.
CHR027	Replay	Rollback proposal	Requests return to a checkpoint or inverse sequence; remains proposal-only until verified.
CHR028	Replay	Deterministic replay	Recomputes events in order and stops at pre/post hash, dependency or adapter divergence.
CHR029	Loop	Loop profile	Declares loop entry, state projection, bound, progress measure, fixed-point and exit semantics.
CHR030	Loop	Loop entry and exit	Typed guard events that enter/leave a looped scenario without hidden teleportation of state.
CHR031	Loop	Fixed-point guard	Accepts a loop closure only when the declared state projection and hashes satisfy the profile.
CHR032	Loop	Progress measure	Well-founded or bounded metric used to prove/observe progress or classify stagnation.
CHR033	Loop	Iteration budget	Maximum turns, events, time, memory and branch expansion before UNKNOWN/STOP.
CHR034	Conflict	Paradox classification	Reason-coded conflict such as missing cause, self-invalidating dependency, duplicate authority or unresolved cycle.
CHR035	Conflict	Same-time conflict	Deterministic priority, compatible merge, commutative batch or explicit conflict event.
CHR036	Order	Event priority profile	Stable ordering keys for proposals sharing one resolution class.
CHR037	Order	Deterministic join	Parallel descendant evaluation joins through recorded order and conflict checks.
CHR038	Uncertainty	Temporal uncertainty interval	Event and observation times carry intervals; ambiguous ordering remains UNKNOWN.
CHR039	Uncertainty	Unknown future state	Uncomputed or underdetermined descendants remain first-class unknowns, not guessed facts.
CHR040	Scenario	Scenario simulation	Pure branch evaluation under bounded models; simulation never edits authoritative history directly.
CHR041	Authority	Timeline patch	Atomic event/branch/dependency/snapshot update with pre/post hashes and invalidation set.
CHR042	Authority	Branch content hash	Canonical branch frontier, event graph and profile digest for identity and cache integrity.
CHR043	Lineage	Timeline lineage graph	Forks, merges, compensations, retractions, adapter versions and replay evidence.
CHR044	Query	Temporal trace query	Why-before/after, ancestors, descendants, first divergence, active branch and unresolved obligations.
CHR045	Query	Branch diff and change report	Stable-ID comparison of events, state, dependencies and unknowns between branches.
CHR046	Packing	Checkpoint plus delta storage	Stores deterministic definitions/checkpoints plus event deltas; external novelty remains explicit.
CHR047	Adapter	Version-control, simulation and narrative adapter	External tools import/export graphs and patches through typed profile and capability negotiation.
CHR048	Certificate	Timeline certificate	Records branch, dependency closure, event order, replay result, loop/merge status, budgets and non-claims.

Claims Ledger

Claim	Disposition	Reason
CHRONOFORGE proves physical time travel.	REJECT	It models timelines, versioned scenarios and time-loop game semantics; it makes no spacetime claim.
Editing the past overwrites the original branch.	REJECT	A verified counterfactual forks from a checkpoint; parent lineage remains immutable.
Equal timestamps imply the same state.	REJECT	Branch, clock domain, event frontier and lineage remain part of identity.
Replay can reconstruct unrecorded external novelty.	REJECT	External events and irreversible transitions must be stored or marked unknown.
Every action has an exact inverse.	REJECT	Reversibility is profile-bound; compensation and irreversible status are distinct.
A loop with repeated coordinates is a valid fixed point.	REJECT	The declared state projection, hashes, dependency context and loop profile must match.
Failure to recompute a future refutes the counterfactual.	REJECT	Budget exhaustion returns UNKNOWN or incomplete descendants.
Branch merging is always possible.	REJECT	Conflicts may yield a non-merge certificate and require manual/domain resolution.
Temporal uncertainty can be discarded after sorting.	REJECT	Overlapping intervals preserve ambiguous order and block claims that depend on it.
A content hash proves causal truth.	REJECT	It proves record integrity, not that the modeled dependency matches reality.
CHRONOFORGE replaces version control, simulation or proof systems.	REJECT	External tools remain adapters; ProofWeave may check scoped certificates.
The cartridge is a complete solution to paradoxes.	REJECT	It supplies explicit profiles, conflict classes and bounded guards, not a universal metaphysics.

High-Impact Application Profiles

Profile	What the cartridge enables
Time-Loop Game Runtime	Reset a world checkpoint while preserving selected knowledge, objects or lineage; explain exactly why the next loop differs.
Paradox Heist	Plan a co-op game where players alter earlier security, power and route events without violating branch, resource or identity rules.
Counterfactual City Lab	Fork CIVITAS incidents, construction and transit schedules; compare routes, services and first divergence.
Branching Cinematic Engine	Drive nonlinear stories whose scenes, choices, clues and consequences are deterministic event graphs rather than fragile flags.
God-Tier Debugger	Replay a runtime, jump to checkpoints, fork a fix, recompute descendants and merge only when state/invariants agree.
Alternate-History Worldbuilder	Keep shared definitions, divergent events, irreversible novelty and lore consistency inside one inspectable branch graph.

Final Formal Definition

FORMAL DEFINITION

UGTOMS CHRONOFORGE 1.0 is a component-scoped temporal cartridge in which clock domains, event and observation intervals, stable event IDs, dependency partial orders, checkpoints, branches, counterfactual proposals, descendant invalidation, reversibility classes, compensating events, loops, fixed-point guards, budgets, merge conflicts, timeline patches, branch hashes and replay lineage are explicit substrate data; past edits fork immutable parent histories; and only a verified atomic timeline patch may create, activate, merge, roll back or retract a branch.

Implementation sequence

15. Freeze schemas for TimelineProfile, EventRecord, BranchRecord, LoopProfile, MergeProfile, TimelinePatch and TimelineCertificate.
16. Implement exact finite event-DAG traversal, checkpoint/replay, fork, invalidation, diff, compensation and non-merge oracle.
17. Ship the CIVITAS signal fork, fixed-point loop, irreversible knowledge and merge-conflict fixtures.
18. Integrate RULEFORGE for time-loop games, CIVITAS for scenario branches and PROOFWEAVE for scoped invariant certificates.
19. Add external version-control, simulation and media-clock adapters only with explicit mapping, uncertainty and replay tests.

RELEASE CONCLUSION

This 1.0 document freezes a domain-facing contract. It is deliberately useful before a production engine exists: implementations can now disagree visibly through typed profiles, reason codes and certificates instead of hiding differences behind the same symbol.